Android 앱 프로젝트 기획 설계서

Midterm Project Planning & Design Document

항목/ Field	학생 작성/ Fill in
앱 이름/ App Name	Personal Expense Tracker
학번/ Student ID	2025621069
이름/ Student Name	Arbin Shrestha
GitHub Repository URL	https://github.com/Arbin0303/Midterm_2025621069_Arbin_Shrestha
작성일/ Date	2026-10-26

Section 1 — 앱 소개 및 개발 목적/ Project Overview

1.1 One-Sentence Description

The Personal Expense Tracker is an Android application that enables users to record daily income and expense transactions, organize them by category, and monitor their monthly financial balance — all stored locally on their device without requiring an internet connection.

1.2 Problem to Solve

Many people struggle to maintain awareness of their daily spending habits. Paper-based records are easily lost, generic spreadsheets are inconvenient on mobile devices, and cloud-based financial services raise privacy concerns or require a constant internet connection. Students and young professionals in particular often reach the end of the month without understanding where their money went.

This application addresses the problem by providing a dedicated, offline, mobile-friendly tool where users can instantly record every financial transaction, assign it to a meaningful category, and review clear summaries of their income, expenses, and remaining balance. Because all data is stored locally using Room Database (SQLite), the information persists across app restarts and device reboots without any network dependency.

1.3 Target Users and Use Cases

Primary users:

- University students managing a monthly allowance or part-time income.

- Young working professionals tracking daily living expenses.
- Anyone who wants a lightweight, privacy-respecting personal finance tool.

Typical use cases:

- A student records a meal expense immediately after eating and assigns it to the "Food" category.
- A user receives their monthly salary and logs it as an income transaction under "Salary."
- At the end of the month, the user opens the Dashboard to see total income, total expenses, and the remaining balance at a glance.
- A user realizes they entered the wrong amount and edits the existing transaction from the Transaction Detail screen.
- A user filters the transaction history by category "Transport" to understand their commuting costs.

1.4 Benefits of Using the Application

- **Offline-first:** No internet connection required; data remains private and always accessible.
- **Quick entry:** A transaction can be recorded in under 30 seconds from anywhere.
- **Visual clarity:** The Dashboard presents a financial summary immediately upon opening the app.
- **Flexible categorization:** Predefined categories (Food, Transport, Education, Shopping, Salary, Entertainment) keep records consistent and filterable.
- **Full control:** Users can edit or delete any record at any time.
- **Development foundation:** The design is intentionally scoped for a beginner-to-intermediate Kotlin developer, making it a realistic foundation for the final project.

Section 2 — 주요 기능 정의 / Core Features

2.1 Feature Table

#	Feature Name	User Action	Expected Result	Android Technology
1	Add Income / Expense Transaction	User taps the "+" FAB on the Dashboard, fills in the form (type, amount, category, date, description), and taps Save	Transaction is validated and saved to Room Database; Dashboard totals update immediately	ViewModel, Room DAO, Kotlin Coroutines, Navigation Component

#	Feature Name	User Action	Expected Result	Android Technology
2	View Transaction History	User navigates to the Transaction History screen	A scrollable list of all transactions is displayed, showing type icon, title, amount, category, and date	RecyclerView, ListAdapter, DiffUtil, Room Flow
3	Categorize Transactions	During transaction entry, user selects a category from a dropdown (Spinner or AutoCompleteTextView)	Transaction is saved with the selected category; category is displayed in the history list and usable as a filter	Spinner / AutoCompleteTextView, Room Entity field
4	View Monthly Summary (Dashboard)	User opens the app or returns to the Dashboard	Displays total income, total expenses, and remaining balance (income − expenses); optionally scoped to the current month	Room DAO aggregate queries (SUM), LiveData / StateFlow, ViewModel
5	Edit and Delete Transactions	User taps a transaction in history, views the detail screen, then taps Edit or Delete	Edit: pre-fills form, user modifies and saves, Room record is updated. Delete: confirmation dialog shown, then record is removed and list refreshes	Room DAO update/delete, AlertDialog, Navigation back-stack
6	Filter Transactions	User selects a filter (by category, date range, or type: Income/Expense) on the History screen	RecyclerView list refreshes to show only matching transactions	Room DAO parameterized queries, LiveData / StateFlow, Chip / Spinner filter UI

2.2 Most Important Feature and Why

The most critical feature is **Add Transaction (Feature 1)**. It is the data-entry point upon which every other feature depends. Without a valid, stored transaction record, the history list is empty, the Dashboard shows zero balances, filtering has nothing to operate on, and edit/delete cannot be reached. Getting this feature right — including input validation, correct coroutine scope, and ViewModel

lifecycle awareness — ensures the entire app functions correctly.

The input validation rules are:

- Amount must be a positive number greater than zero.
- Transaction type (INCOME or EXPENSE) must be selected.
- Category must be selected.
- Date must be provided (default to today if not changed by the user).
- Title must not be blank.

2.3 RecyclerView Usage Plan

`TransactionHistoryFragment` uses a `RecyclerView` with a custom `TransactionAdapter` (extending `ListAdapter\<TransactionEntity, TransactionAdapter.ViewHolder\>`) to display the list of transactions. Each item card shows:

- A color-coded icon indicating INCOME (green) or EXPENSE (red).
- Transaction title.
- Amount (formatted with currency symbol).
- Category label.
- Date (formatted as "dd MMM yyyy").

`DiffUtil.ItemCallback` is implemented to calculate list differences efficiently, enabling smooth animations when items are added, updated, or removed without redrawing the entire list.

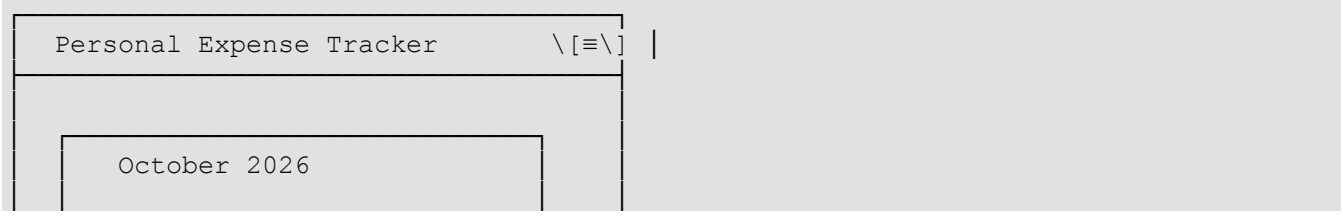
The `DashboardFragment` also uses a smaller `RecyclerView` to display the five most recent transactions as a "Recent Activity" preview, using the same adapter with a limited dataset.

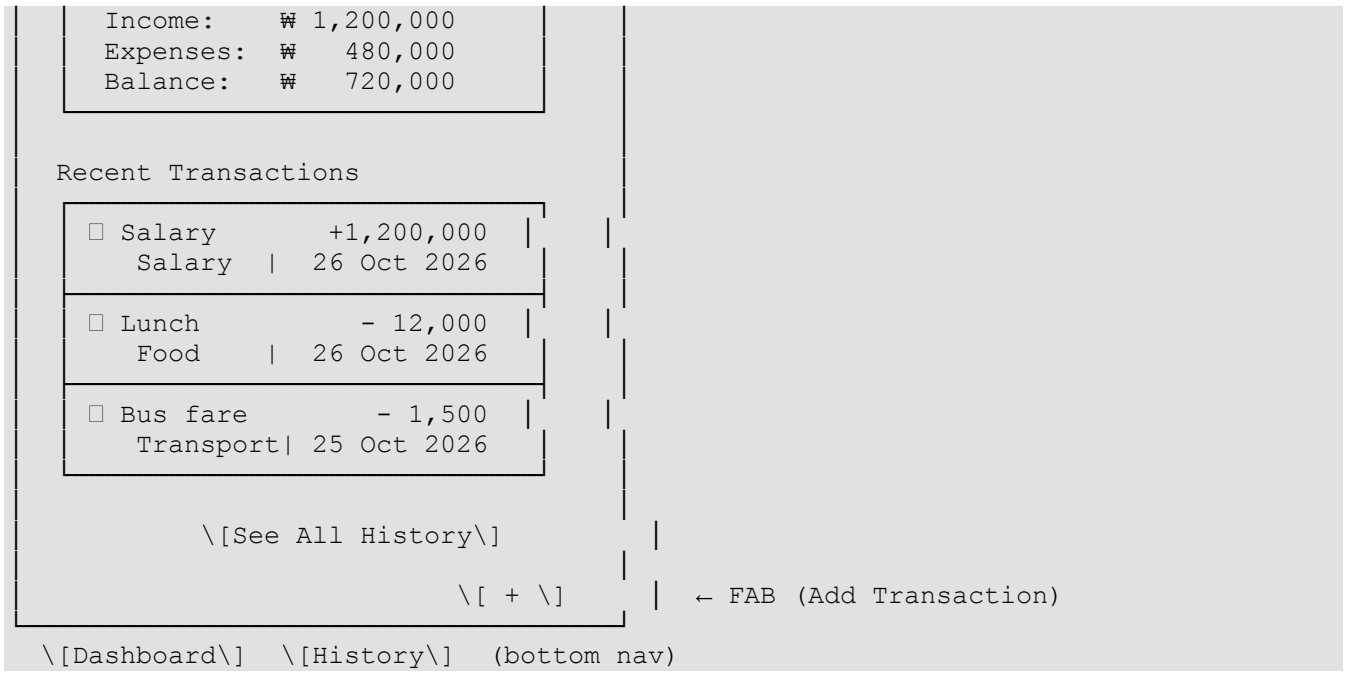
Section 3 — 화면 구성 설계/ Screen Wireframes

Four primary screens are designed. All screens are implemented as Fragments hosted in a single `MainActivity`, connected through the Navigation Component.

Screen 1 — DashboardFragment

Purpose: Entry point of the app. Provides an at-a-glance financial summary and quick access to all other screens.



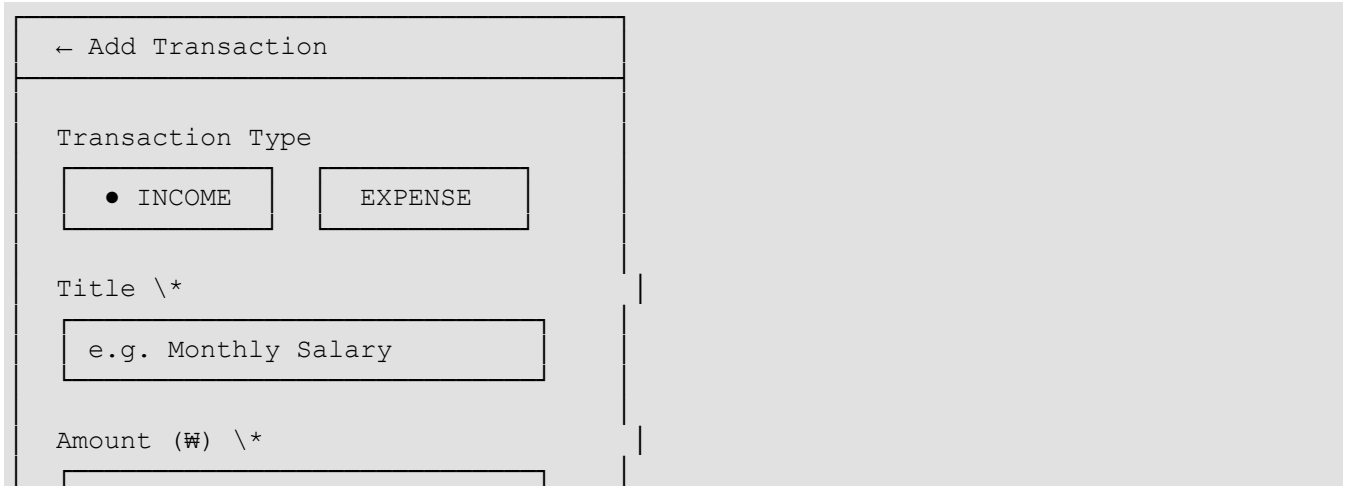


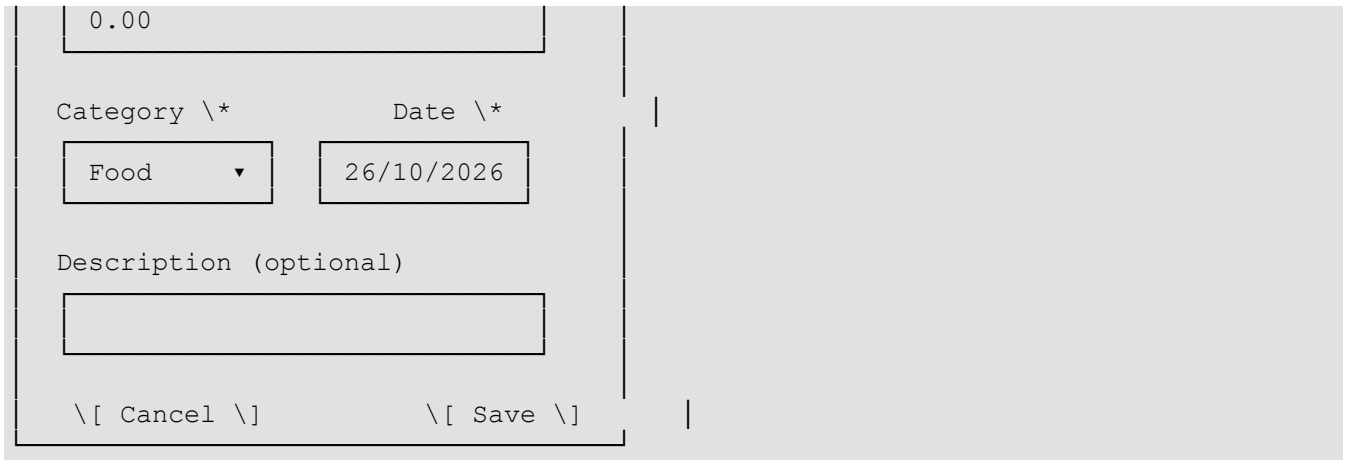
Main UI Elements:

- `CardView` summary card: Income, Expenses, Balance TextViews.
- `RecyclerView` (recent transactions, max 5 items).
- `FloatingActionButton` (FAB) — navigates to `AddTransactionFragment`.
- `Button` / `TextView` "See All History" — navigates to `TransactionHistoryFragment`.
- `BottomNavigationView` with Dashboard and History tabs.

Screen 2 — AddTransactionFragment

Purpose: Allows the user to enter all details for a new income or expense transaction and save it to the database.



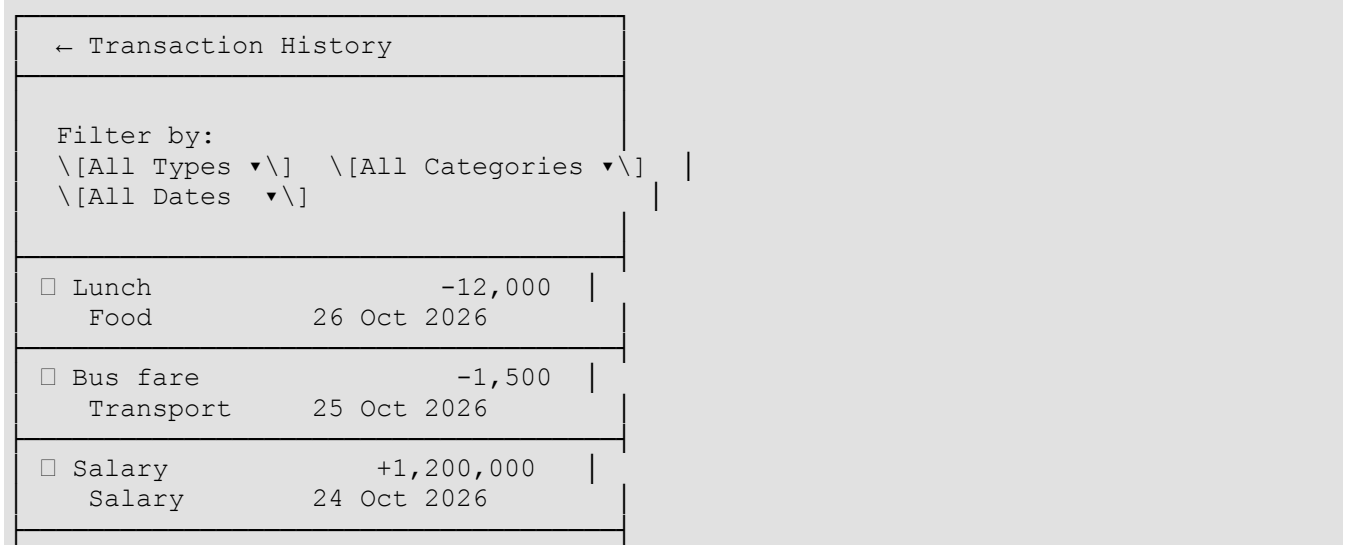


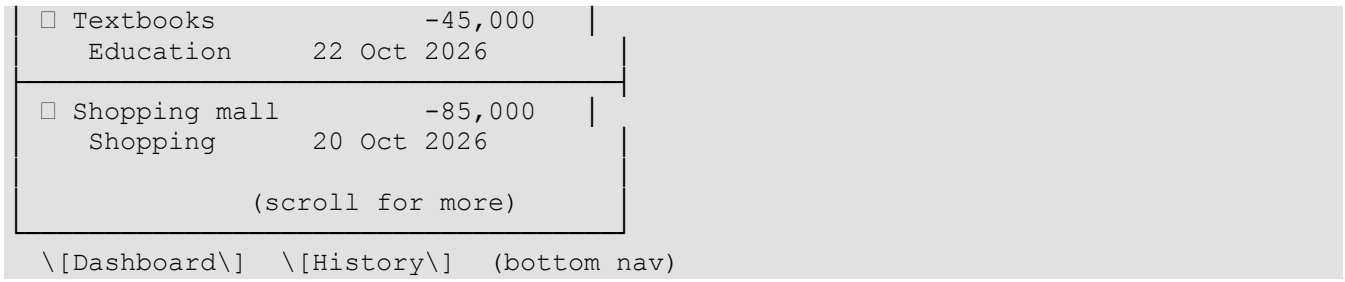
Main UI Elements:

- `RadioGroup` / `ToggleButton` pair for Income / Expense type selection.
- `EditText` for Title (required).
- `EditText` (InputType: numberDecimal) for Amount (required).
- `Spinner` or `AutoCompleteTextView` for Category: Food, Transport, Education, Shopping, Salary, Entertainment.
- `TextView` + `DatePickerDialog` for Date selection (defaults to today).
- `EditText` (multiLine) for optional Description.
- `Button` Save — triggers ViewModel insert and pops back-stack.
- `Button` Cancel — pops back-stack without saving.

Screen 3 — TransactionHistoryFragment

Purpose: Displays the complete list of all recorded transactions with filter controls.



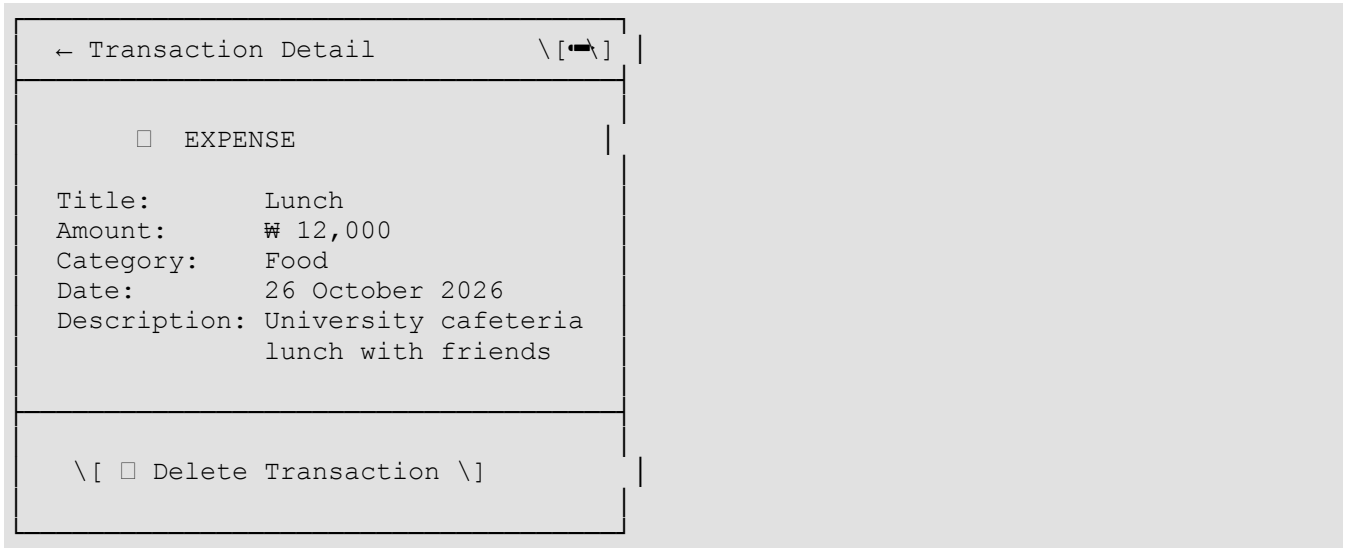


Main UI Elements:

- `Spinner` (Type filter): All, Income, Expense.
- `Spinner` (Category filter): All, Food, Transport, Education, Shopping, Salary, Entertainment.
- `Spinner` (Date filter): All Time, This Month, This Week.
- `RecyclerView` with `TransactionAdapter` — each row tappable; navigates to `TransactionDetailFragment`.
- Empty state `TextView`: "No transactions found. Tap + to add one."

Screen 4 — TransactionDetailFragment

Purpose: Displays the full details of a single transaction and provides options to edit or delete it.



Main UI Elements:

- Type indicator icon (green = INCOME, red = EXPENSE).
- `TextView` fields: Title, Amount, Category, Date, Description.
- Edit icon (`ImageButton`) in the toolbar — navigates to `AddTransactionFragment` pre-filled with the selected transaction's data (using Safe Args to pass `transactionId`).
- `Button` "Delete Transaction" — triggers an `AlertDialog` confirmation ("Are you sure you

want to delete this transaction? This action cannot be undone."). On confirm: calls ViewModel delete, then pops back-stack to History.

Screen 5 (Optional) — MonthlyReportFragment

Purpose: Provides a month-by-month breakdown of income and expenses, showing category totals as a simple text-based summary.

```
┌──────────────────────────────────────┐
│  ← Monthly Report                    │
├──────────────────────────────────────┤
│                                      │
│  \[ \< October 2026 \> \]            │
│                                      │
│  Income:   ₩ 1,200,000               │
│  Expenses: ₩   480,000               │
│  Balance:  ₩   720,000               │
│                                      │
│  Expenses by Category:               │
│  ┌──────────────────────────────┐    │
│  │ Food          ₩ 142,000      │    │
│  │ Transport     ₩  38,500      │    │
│  │ Education     ₩  45,000      │    │
│  │ Shopping      ₩  85,000      │    │
│  │ Entertainment ₩ 169,500      │    │
│  └──────────────────────────────┘    │
│                                      │
│  Income by Category:                 │
│  ┌──────────────────────────────┐    │
│  │ Salary      ₩ 1,200,000      │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
  \[Dashboard\]  \[History\]  (bottom nav)
```

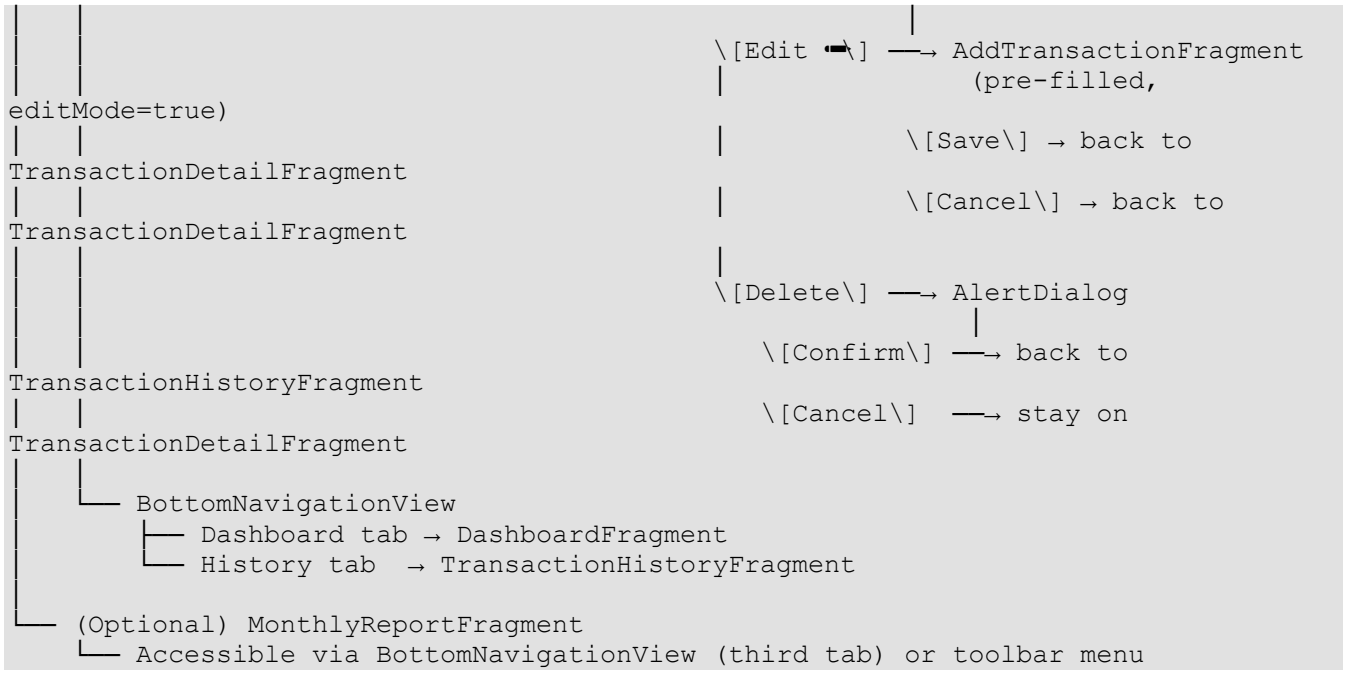
Main UI Elements:

- Month navigation buttons (previous/next month).
- Summary `TextViews` for total income, expenses, balance.
- `RecyclerView` or static `LinearLayout` for category breakdown.

Section 4 — 화면 이동 흐름 / Navigation Flow

4.1 Navigation Diagram

```
MainActivity (NavHostFragment)
│
├── DashboardFragment  \[Start Destination\]
│    ├── FAB (+) ──────────────────────────────→ AddTransactionFragment
│    │                                                  │
│    │                                       \[Save\] ──→ back to DashboardFragment
│    │                                       \[Cancel\] → back to DashboardFragment
│    │
│    ├── "See All History" button ──────────────→ TransactionHistoryFragment
│    │                                                  │
│    │                                    \[row tap\] ──→ TransactionDetailFragment
```



4.2 Navigation Actions Explained

A. Adding a new transaction: The user taps the FAB (+) on the Dashboard. The Navigation Component executes the action `action_dashboard_to_addTransaction`. The `AddTransactionFragment` opens in a new destination. After the user fills in the form and taps Save, the ViewModel calls the Repository insert method. Upon successful completion (observed via StateFlow), `findNavController().popBackStack()` is called, returning the user to the Dashboard. The Dashboard's LiveData/Flow observing the summary query automatically reflects the updated totals.

B. Viewing and managing transaction history: The user navigates to `TransactionHistoryFragment` via the bottom navigation or the "See All History" button. The RecyclerView is populated from a Flow query in the ViewModel. Tapping a row passes the `transactionId` via Safe Args to `TransactionDetailFragment`. From the detail screen, the Edit button passes the same `transactionId` to `AddTransactionFragment` in edit mode, where the form fields are pre-filled by loading the entity from the database.

C. Deleting a transaction: On `TransactionDetailFragment`, tapping Delete shows an `AlertDialog`. If the user confirms, `ViewModel.deleteTransaction(id)` is called via a coroutine, and `popBackStack()` returns the user to `TransactionHistoryFragment`. The RecyclerView updates automatically because the DAO returns a `Flow<List<TransactionEntity>>`, and LiveData/StateFlow propagates the change reactively.

D. Editing a transaction: Edit mode in `AddTransactionFragment` is indicated by a non-null `transactionId` argument (Safe Args). The ViewModel loads the existing entity, populates the form fields, and on Save calls `ViewModel.updateTransaction()` instead of insert. After

saving, the user is popped back to `TransactionDetailFragment` with refreshed data.

Section 5 — Room SQLite 데이터베이스 설계/ Database Design

5.1 Database Name

`expense\_tracker\_database`

5.2 Primary Table: `transactions`

Column Name	Data Type	Key / Constraint	Description	Example Value
`transactionId`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each transaction, auto-assigned by Room.	1, 2, 3, …
`title`	TEXT	NOT NULL	Short descriptive name of the transaction.	"Monthly Salary"
`amount`	REAL	NOT NULL, > 0	Monetary value of the transaction (always positive).	1200000.0, 12000.0
`type`	TEXT	NOT NULL, CHECK (type IN ('INCOME','EXPENSE'))	Indicates whether money came in or went out.	"INCOME", "EXPENSE"
`category`	TEXT	NOT NULL	User-assigned category for grouping and filtering.	"Food", "Salary"
`date`	INTEGER	NOT NULL	Transaction date stored as Unix epoch (milliseconds since 1970-01-01). Enables easy range queries and consistent sorting.	1761446400000
`description`	TEXT	NULLABLE	Optional free-text note about the transaction.	"Lunch at cafeteria", NULL

SQL equivalent:

```
CREATE TABLE IF NOT EXISTS transactions (
    transactionId   INTEGER PRIMARY KEY AUTOINCREMENT,
    title           TEXT    NOT NULL,
```

```
    amount          REAL    NOT NULL,
    type            TEXT    NOT NULL,
    category        TEXT    NOT NULL,
    date            INTEGER NOT NULL,
    description     TEXT
);
```

5.3 Example Records

transactionId	title	amount	type	category	date	description
1	Monthly Salary	1200000.0	INCOME	Salary	1761446400000	October salary deposit
2	Lunch	12000.0	EXPENSE	Food	1761532800000	University cafeteria
3	Bus fare	1500.0	EXPENSE	Transport	1761446400000	NULL
4	Textbooks	45000.0	EXPENSE	Education	1761273600000	Android programming book
5	Weekend mall	85000.0	EXPENSE	Shopping	1761100800000	Clothes and accessories

5.4 Primary Key Explanation

`transactionId` is the primary key. It uniquely identifies each transaction row and is auto-incremented by Room, so the application never needs to assign IDs manually. Room uses this column in DAO methods such as `updateTransaction(entity)` and `deleteById(id)` to target the correct record.

5.5 Date Storage Strategy

The `date` column stores a **Long (Unix epoch in milliseconds)**. This approach is chosen because:

- Room stores it as a single INTEGER with no parsing overhead.
- Kotlin's `System.currentTimeMillis()` produces this value directly.
- Range queries ("all transactions this month") can be expressed as simple `WHERE date BETWEEN startMs AND endMs`.
- Display formatting (e.g., "26 Oct 2026") is handled in the UI layer using `SimpleDateFormat` or `java.time.LocalDate`.

5.6 Why Room Database is Appropriate

- Room is the recommended Android abstraction for SQLite. It provides compile-time SQL verification, reducing runtime crashes from query errors.

- The `@Entity`, `@Dao`, and `@Database` annotation system integrates cleanly with Kotlin data classes.
- Room's native support for `Flow\<List\<T\>\>` return types from DAO methods enables reactive UI updates without manual refresh logic.
- Room handles database versioning and migration through `@Database(version = N)` and `Migration` objects.
- For a single-user, offline, student-scope project, one table is sufficient and keeps the design simple and maintainable.

5.7 Additional Table (Future Consideration)

A second table `categories` is not required for the initial implementation because the six categories (Food, Transport, Education, Shopping, Salary, Entertainment) can be defined as a constant list in the application code. If the final project requires user-defined custom categories, a `categories` table can be added with columns `categoryId` (INTEGER PK), `name` (TEXT NOT NULL), and `type` (TEXT NOT NULL — INCOME or EXPENSE). This is noted as a potential extension, not part of the current scope.

Section 6 — Room 및 데이터 처리 계획 / Room and Data Processing Plan

6.1 Component Roles

Component	File Name	Planned Responsibility
TransactionEntity	`TransactionEntity.kt`	Kotlin data class annotated with `@Entity(tableName = "transactions")`. Each property maps to a database column. The `transactionId` property is annotated with `@PrimaryKey(autoGenerate = true)`.
TransactionDao	`TransactionDao.kt`	Interface annotated with `@Dao`. Declares all database operations: `insertTransaction`, `updateTransaction`, `deleteTransaction`, `getAllTransactions`, `getTransactionById`, `getTransactionsByTypeAndCategory`, `getTotalByType`.
AppDatabase	`AppDatabase.kt`	Abstract class extending

Component	File Name	Planned Responsibility
		RoomDatabase, annotated with @Database(entities = \[TransactionEntity::class\], version = 1). Provides a singleton instance and exposes transactionDao().
TransactionRepository	TransactionRepository.kt	Mediates between ViewModels and the DAO. Exposes suspend functions (insert, update, delete) and Flow properties (allTransactions, filteredTransactions) so that ViewModels have no direct dependency on Room.
TransactionViewModel	TransactionViewModel.kt	Extends AndroidViewModel. Holds a reference to TransactionRepository. Exposes StateFlow\<List\<TransactionEntity\>\> for the history screen and StateFlow\<SummaryData\> for the dashboard. Launches coroutines using viewModelScope.
DashboardFragment	DashboardFragment.kt	Observes dashboardSummary StateFlow from the ViewModel to update Income, Expenses, and Balance TextViews. Observes recentTransactions Flow to populate the short RecyclerView. Sets up FAB click listener to navigate to AddTransactionFragment.
AddTransactionFragment	AddTransactionFragment.kt	Reads form inputs (type, title, amount, category, date, description). Performs client-side validation. In insert mode: calls ViewModel.insert(entity). In edit mode (transactionId passed via Safe Args): loads entity, pre-fills form, calls

Component	File Name	Planned Responsibility
		`ViewModel.update(entity)`.
TransactionHistoryFragment	`TransactionHistoryFragment.kt`	Observes a filterable Flow of transactions from the ViewModel. Sets up the RecyclerView and adapter. Listens to filter Spinner selections to trigger filtered DAO queries via the ViewModel.
TransactionDetailFragment	`TransactionDetailFragment.kt`	Receives `transactionId` via Safe Args. Loads the entity via `ViewModel.getById(id)`. Displays all fields. Handles Edit navigation (passing id to `AddTransactionFragment`) and Delete with AlertDialog confirmation.

6.2 Data Flow Diagram

```
User types data in AddTransactionFragment
        |
        ▼
Client-side validation (amount \> 0, title not blank, type and category selected)
        |  (if invalid → show error message, stop here)
        |  (if valid ↓)
        ▼
TransactionViewModel.insertTransaction(entity)   \[viewModelScope.launch\]
        |
        ▼
TransactionRepository.insertTransaction(entity)  \[suspend function\]
        |
        ▼
TransactionDao.insertTransaction(entity)         \[@Insert suspend fun\]
        |
        ▼
Room writes to SQLite  (expense\_tracker\_database / transactions table)
        |
        ▼
TransactionDao.getAllTransactions()              \[returns
Flow\<List\<TransactionEntity\>\>\]
        |
        ▼
TransactionRepository exposes the Flow ──→ TransactionViewModel collects it
        |
        ▼
TransactionHistoryFragment / DashboardFragment
observe StateFlow ──→ RecyclerView.submitList() / TextViews updated
```

6.3 Kotlin Coroutines and Flow Usage

Coroutines for asynchronous operations: All Room DAO write operations (`insert`, `update`, `delete`) are declared as `suspend` functions. The ViewModel launches them inside `viewModelScope.launch(Dispatchers.IO)` so they execute on a background thread, keeping the UI thread responsive. This prevents the "Application Not Responding" (ANR) error that would occur if database writes happened on the main thread.

```
// TransactionViewModel.kt (proposed)
fun insertTransaction(entity: TransactionEntity) \{
    viewModelScope.launch \{
        repository.insertTransaction(entity)
    \}
\}
```

Flow for reactive read operations: Read queries (e.g., `getAllTransactions()`) return `Flow\<List\<TransactionEntity\>\>`. The ViewModel converts this into a `StateFlow` using `.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())`. Fragments collect this StateFlow using `lifecycleScope.launch \{ repeatOnLifecycle(Lifecycle.State.STARTED) \{ ... \} \}`. Whenever the database changes, Room automatically emits a new list, and the Fragment updates the RecyclerView without any manual refresh call.

```
// TransactionDao.kt (proposed)
@Query("SELECT \* FROM transactions ORDER BY date DESC")
fun getAllTransactions(): Flow\<List\<TransactionEntity\>\>

@Query("SELECT SUM(amount) FROM transactions WHERE type = :type")
fun getTotalByType(type: String): Flow\<Double?\>
```

Summary computation for Dashboard: The ViewModel collects both `getTotalByType("INCOME")` and `getTotalByType("EXPENSE")` as separate Flows, combines them using `combine \{ income, expense -\> SummaryData(income, expense, income - expense) \}`, and exposes the result as a single `StateFlow\<SummaryData\>` for the Dashboard to observe.

Section 7 — 개발 일정 및 제출 계획 / Development Schedule

Stage	Period	Tasks	Completion Criteria
Stage 1: Setup & UI Design	Weeks 9–10	• Create Android Studio project with Kotlin and MVVM structure. • Set up `build.gradle` dependencies: Room, Navigation, RecyclerView, Lifecycle/ViewModel, Material Design. •	All four screens display correctly in the emulator. Navigation between screens works without crashes. GitHub repository is created with initial commit.

Stage	Period	Tasks	Completion Criteria
		Design all four Fragment XML layouts: Dashboard, AddTransaction, TransactionHistory, TransactionDetail. • Create navigation graph (`nav\_graph.xml`) with all destinations and actions. • Add `BottomNavigationView` to `MainActivity`. • Initialize Git repository and push to GitHub.	
Stage 2: Navigation & RecyclerView	Weeks 11–12	• Implement `TransactionAdapter` with `ListAdapter` and `DiffUtil`. • Connect RecyclerView in `TransactionHistoryFragment` with static/mock data to verify layout. • Implement filter Spinner listeners and UI state changes. • Connect FAB on Dashboard to `AddTransactionFragment` via Navigation. • Implement Safe Args for passing `transactionId` between History → Detail → Edit. • Add `DatePickerDialog` to AddTransaction screen.	RecyclerView scrolls and displays mock data. Row taps open the detail screen with correct data passed. Filter controls update the visible list. Date picker dialog works correctly.
Stage 3: Room Database & CRUD	Weeks 13–14	• Define `TransactionEntity` data class. • Implement `TransactionDao` with all required queries	CRUD operations persist data across app restarts. Dashboard totals reflect real database records. Deleting a transaction removes it from the list.

Stage	Period	Tasks	Completion Criteria
		(insert, update, delete, getAll, getById, getFiltered, getTotalByType). • Implement `AppDatabase` singleton. • Implement `TransactionRepository`. • Implement `TransactionViewModel` with StateFlows. • Connect Fragments to ViewModel: save from AddTransaction, load list in History, load detail in DetailFragment, delete with confirmation. • Implement dashboard summary (income, expense, balance).	Editing updates the record correctly.
Stage 4: Testing, Polish & Submission	Weeks 15–16	• Test all user flows on emulator and physical device (if available). • Fix crashes and edge cases: empty amount, invalid input, delete last item. • Improve UI polish: empty state messages, loading indicators, input error labels. • Verify RecyclerView scroll performance. • Add AI use disclosure to submission document. • Finalize GitHub repository: clean commit history, README with app description and screenshot. • Export document to PDF with correct filename.	App runs without crashes for all flows. All required assignment checklist items are confirmed. GitHub repository URL is valid and accessible. PDF is correctly named and submitted.

Section 8 — 최종 점검/ Submission Checklist

#	Checklist Item	Status
☑	App purpose and target users described (Section 1)	Included
☑	At least 4 core features defined with technology (Section 2, 6 features)	Included
☑	At least 4 screen designs with wireframes and UI elements (Section 3, 5 screens)	Included
☑	Navigation flow diagram with transition explanations (Section 4)	Included
☑	RecyclerView usage plan described (Section 2.3)	Included
☑	Room database design with table name, column names, data types, and primary key (Section 5)	Included
☑	Development schedule across Weeks 9–16 with completion criteria (Section 7)	Included
☑	GitHub Repository URL	https://github.com/Arbin0303/Midterm_2025621069_Arbin_Shrestha
☑	PDF filename format: `Midterm\_StudentID\_Name.pdf`	`Midterm_2025621069_ArbinShrestha`
☑	AI use disclosure	See below

AI Use Disclosure / AI 활용 내용

This planning document was prepared with the assistance of an AI tool (Kiro / Claude). The AI was used to help structure the document outline, draft section text, design ASCII wireframes, and suggest technically consistent database schema and architecture patterns based on the project requirements provided by the student.

The student is responsible for reviewing and understanding all content in this document, replacing placeholder fields (Student ID, Name, GitHub URL) with their actual information, and being prepared

to explain the design decisions in any academic setting.

All design decisions — including the choice of app topic (Personal Expense Tracker), features, screen layouts, database schema, and development schedule — were determined by and remain consistent with the requirements supplied by the student.

End of Document

Prepared: 2026-10-26 Platform: Android | Language: Kotlin | IDE: Android Studio Architecture: MVVM + Repository | Database: Room (SQLite) | Navigation: Navigation Component